

2026年1月29日 工程

深入了解 OpenAI 的内部数据智能体

作者：Bonnie Xu、Aravind Suresh 和 Emma Tang

正在加载...



数据为系统如何学习、产品如何演进以及企业如何做出选择提供动力。但要快速、准确地获取答案，并结合相关上下文，往往比想象中更为困难。随着 OpenAI 业务范围的扩展，为了简化这项工作，我们建立了**专属定制内部 AI 数据智能体**，通过 OpenAI 的平台进行探索和推理。

我们的智能体是一款定制内部专用工具（非外部产品），围绕 OpenAI 的数据、权限和工作流而构建。我们将展示 OpenAI 如何构建和使用这一工具，以帮助大家了解 AI 如何以切实可行的方式，支持团队完成日常工作。我们用于构建和运行这一智能体的 OpenAI 工具（[Codex](#)、[我们的 GPT-5 旗舰模型](#)、[Evals API](#) 和 [Embeddings API](#)）与我们面向全球各地开发人员提供的工具相同。

我们的数据智能体可支持员工在几分钟（而非几天）内从问题中提取洞察数据。这降低了所有职能部门（而不仅仅是我们的数据团队）提取数据和进行细致分析的门槛。如今，OpenAI 的工程、数据科学、市场进入、财务和研究团队都依赖智能体来解答**高价值数据问题**。例如，它可以通过直观的自然语言格式，回答如何评估发布流程和理解业务运行状况等问题。该智能体结合了 Codex 驱动的表格级知识，以及产品和组织的背景信息。其持续学习记忆系统意味着它能够不断优化。

On October 6, 2025, what was our ChatGPT WAU?
Compare against our ChatGPT WAU on the last dev day in 2023. Round both to the nearest 100M.

Worked for 1m 22s ›

ChatGPT WAU on October 6, 2025 (rounded to nearest 100M): ≈ 800M

ChatGPT WAU on the last DevDay 2023 (Nov 6, 2023; rounded to nearest 100M): ≈ 100M

Mini comparison (using the rounded figures):

- Change: ≈ +700M WAU
- Multiple: ≈ 8x higher on 2025-10-06 vs 2023-11-06

(WAU here is the standard ChatGPT WAU as-of the reporting date; I'm only sharing the values rounded to the nearest 100M, per your request.)

OpenAI

代码增强型数据上下文和自学习功能为何如此重要，以及我们在此过程中汲取的经验教训。

为什么我们需要定制工具

OpenAI 的数据平台为工程、产品和研究部门的 **3,500 多名内部用户** 提供服务，涵盖 **70,000 个数据集** 中超过 **600 PB** 的数据。在此规模下，寻找合适的表格可能是分析过程中最耗时的环节之一。

正如一位内部用户所解释的：

“我们有很多非常相似的表格，我曾耗费大量时间去厘清它们的不同之处，以及具体应选择哪一个表格。有的表格包括已注销用户，有的则不包括这些用户。有的表格存在重叠字段，很难分辨具体内容。”

即使选择了正确的表格，生成正确的结果也并非易事。分析师必须对表格数据和表格关系进行推理，以确保正确应用转换和筛选功能。常见的故障模式 — 多对多连接、筛选条件下推错误和未处理的空值，都可能会在不知不觉中输出无效结果。鉴于 OpenAI 庞大的组织架构，分析师不应将时间浪费在调试 SQL 语义或查询性能上：他们的重点应放在定义指标、验证假设和制定数据驱动的决定。

```
145 -- 6) Attach customer geography; compute monthly aggregates
146 order_enriched AS (
147     SELECT
148         r.*,
149         g.c_name,
150         g.c_mktsegment,
151         g.c_acctbal,
152         COALESCE(g.region_name, 'UNKNOWN') AS region_name,
153         COALESCE(g.nation_name, 'UNKNOWN') AS nation_name,
154         DATE_TRUNC('MONTH', r.o_orderdate) AS order_month
155     FROM order_rollup r
156     LEFT JOIN cust_geo g
157         ON g.c_custkey = r.o_custkey
158 ),
159
160 monthly_segment AS (
161     SELECT
162         order_month,
163         region_name,
164         c_mktsegment,
165         COUNT(*) AS orders,
166         SUM(order_gross_revenue) AS gross_revenue,
167         SUM(order_gross_revenue_with_tax) AS gross_revenue_with_tax,
168         AVG(avg_ship_to_receipt_days) AS avg_ship_to_receipt_days
169     FROM order_enriched
170     GROUP BY
171         order_month,
172         region_name,
173         c_mktsegment
174 )
175
176 SELECT
177     order_month,
178     region_name,
179     c_mktsegment,
180     orders,
181     gross_revenue,
182     gross_revenue_with_tax,
183     avg_ship_to_receipt_days
```

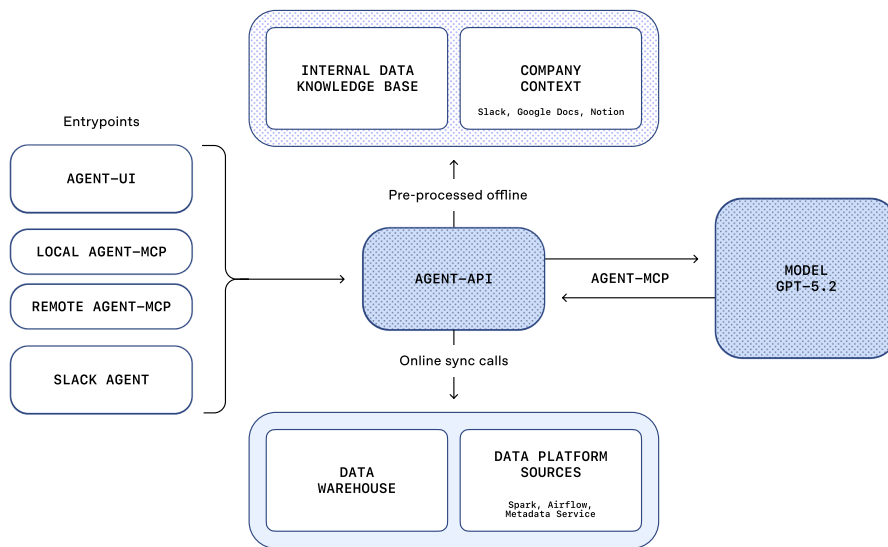
这条 SQL 语句长达 180 多行。要确认我们是否连接正确的表格并查询相关列，并非易事。

工作原理

让我们逐步了解 OpenAI 的智能体及其如何整理上下文并不断自我完善。

我们的智能体由 **GPT-5.2** 驱动，旨在通过 OpenAI 的数据平台进行推理。无论员工在何处办公，都可以使用这一工具：它能够充当 Slack 智能体、通过 Web 界面、嵌入集成开发环境 (IDE)、经由 MCP 连接的 Codex CLI 以及直接通过 MCP 连接器在 OpenAI 的内部 ChatGPT 应用中调用。

How the data agent works



用户可以提出复杂的开放式问题，这通常需要多轮人工探索。以使用测试数据集的提示为例：“*For NYC taxi trips, which pickup-to-dropoff ZIP pairs are the most unreliable, with the largest gap between typical and worst-case travel times, and when does that variability occur?*”

从理解问题到探索数据、运行查询和汇总结果，**智能体可负责执行端到端分析。**

分析 数据 查询 图表

OpenAI

worst-case duration), and when?

Worked for 6m 7s >

Using `samples.nyctaxi.trips` (~21,932 trips; **2016-01-01 → 2016-02-29**), I measured "unreliability" as the **spread = p95(duration) - p50(duration)** for each pickup→dropoff ZIP.

- **Typical** = median trip duration (p50)
- **Worst-case** = 95th percentile duration (p95)
- Filters: duration in **[1, 180] minutes**, ZIP pairs with **≥ 20 trips**
- "When?" below is shown as the **pickup local datetime (NYC time)** of the single longest trip observed for that ZIP pair (because the sample is too small to robustly estimate hour-of-week tail behavior per pair).

智能体对问题的答复。

智能体的一项核心优势在于其解决问题的推理能力。智能体并非遵循固定的脚本，而是会自行评估流程进展。如果某个中间结果出现异常（例如，由于错误的连接或筛选操作导致行数为零），智能体就会调查出现非问题的具体环节，调整其方法，并再次尝试。在此过程中，它会保留完整的上下文，并在各步骤之间传递学习成果。这一**闭环式自主学习过程**可将迭代任务从用户转移到智能体自身，从而更快输出结果，并持续提供质量远高于手动工作流的分析数据。

Worked for 6m 7s ▾

Planning duration analysis

I'm thinking about computing the p50 and p95 duration based on pickup and dropoff zip codes and time buckets like hour of day, day of week, and month. The user wants to know when unreliability peaks, so I'll note the time buckets with the highest spread. Since the NYC taxi dataset is large, I should filter for a recent period—maybe the last 12 months to capture seasonality. I'll need to identify the right table names first. Let's start with a full internal search for the data!

Searching internal catalogs/docs for NYC Taxi trip tables and any ZIP-enriched fields or zone→ZIP mapping, so we can compute percentile-based duration spread by pickup→dropoff ZIP and time bucket.

```
full_internal_search {
  "query": "NYC taxi trips table pickup_zip dropoff_zip duration"
}
```

Inspecting dataset schema

I've come across the example query `samples.nyctaxi.trips`, and it seems to point to the `samples.nyctaxi.trips` table, which is likely available in the Databricks sample dataset. I wonder if it includes columns like pickup_zip or dropoff_zip. To clarify this, I'll need to inspect the schema, so I'm planning to use `mcp_keeper.get_table_schema` to check it out. Let's see what details I find!

Found a promising built-in dataset: `samples.nyctaxi.trips`. Next I'll inspect its schema to confirm it has pickup/dropoff ZIPs (or lat/long we can map to ZIP) and trip timestamps/duration.

```
get_table_schema {
  "table_name": "samples.nyctaxi.trips",
  "query_limit": 5
}
```

Calculating trip duration spreads

I'm considering showing the overall typical and worst-case scenarios while also examining other times. I'll compute the global spread across all times using p50 and p95. For each pair, I want to identify the hour of the week with the maximum spread and value.

I'll write a query with a Common Table Expression (CTE) where I derive the necessary fields and aggregate data by pickup and dropoff zip codes, day of the week, and hour. After filtering and ranking pairs, I'll return the top 20 pairs based on their maximum spread!

Streamlining trip analysis

I'm thinking it might get messy, so I'll simplify the task. For each top pair, I can find the pickup day of the week and hour for trips that exceed the p95 threshold, identifying the most frequent occurrences. I also want to compute the count of tail trips per pair, determining the maximum tail count. I aim to say which pairs are the most unreliable and when their worst-case trips occur. The plan includes cleaning the data and crafting a query to return the top 15 pairs with relevant statistics. Let's get started!

Time-of-week buckets are very sparse in this sample dataset (max 5 trips per ZIP-pair-hour). I'll instead rank ZIP pairs by overall `p95 - p50` duration spread, then find *when* each pair's worst-case tail (≈p95) most often occurs (weekday + hour).

智能体可凭借推理能力，识别最不可靠的纽约市出租车上车点-下车点配对数据。

该智能体已涵盖完整的分析工作流：发现数据、运行 SQL、发布笔记及报告。它能够理解企业内部知识，通过网络搜索获取外部信息，

<https://openai.com/zh-Hans-CN/index/inside-our-in-house-data-agent/>

4/13

上下文决定一切

高质量的答案取决于**丰富、准确的上下文**。在缺乏上下文的情况下，即使是强大的模型也可能会输出错误结果，例如严重误估用户数量或曲解内部术语。

没有记忆能力的智能体，无法有效进行查询。

智能体的记忆可通过定位正确的表格，加快查询速度。

为了避免这些故障模式，我们围绕**多层上下文构建智能体**，并以**OpenAI 的数据和机构知识为基础**。

第 1 层：表格使用情况

- **元数据基础：**智能体依赖模式元数据（列名和数据类型）来指导 SQL 编写，并使用表格沿革（例如，上游和下游表格关系）来理解不同表格之间的上下文。
- **查询推理：**采集历史查询有助于智能体理解如何编写查询，以及哪些表格通常相互关联。

第 2 层：人工注释

- **精选描述：**由领域专家精心整理的表格和列相关描述，用于记录意图、语义、业务含义，以及无法从模式或过往查询中轻易推断的已知注意事项。

OpenAI

来源。

第 3 层：Codex 增强

- 通过推导表格的代码级定义，智能体能够更深入地理解数据的实际内容。
 - 关于表格中存储的内容以及如何根据分析事件中得出的细微差别提供额外的信息。例如，它可以提供关于值的唯一性、表格数据更新频率、数据范围（例如，如果表格排除某些字段，即标识其具备相应的粒度级别）等上下文。
- 通过展示如何在 Spark、Python 和其他数据系统中使用 SQL 以外的表格，提供更丰富的使用情况上下文。
- 这意味着，智能体可以区分内容相似但存在关键差异的表格。例如，它可以判断某个表格是否仅包含第一方 ChatGPT 流量。该上下文还会自动刷新，因此无需手动维护即可保持更新。

第 4 层：机构知识

- 智能体可以访问 Slack、Google Docs 和 Notion，这些平台可记录关键的企业背景信息，例如产品发布、可靠性事件、内部代号和工具，以及关键指标的规范定义和计算逻辑。
- 这些文档会被采集、嵌入，并与元数据和权限一起存储。检索服务可在运行时处理访问控制和缓存，支持智能体高效且安全地获取这些信息。

第 5 层：记忆

- 当智能体收到更正信息或发现某些数据问题存在细微差别时，它能够保存这些学习结果以供后续使用，从而在与用户交互的过程中持续改进。
 - 因此，未来的答案将以更准确的基线为切入点，而非反复处理相同的问题。
 - 记忆的目标是保留并复用那些不易察觉的更正信息、过滤条件和约束限制，这些内容对于维持数据的正确性至关重要，但仅凭其他层级难以有效推断。
 - 例如，在某个案例中，智能体不知道如何筛选特定的分析实验（它需要依赖与实验门中定义的特定字符串进行匹配）。在此过程中，记忆至关重要，因为这一能力可确保智能体正确筛选相关信息，而非盲目尝试进行字符串匹配。
- 当你向智能体提出更正意见，或者当其从对话中查找学习内容时，它会提示你保存记忆，以备后续使用。
 - 用户也可以手动创建和编辑记忆。
 - 记忆的范围涵盖全局和个人层面，而智能体所配备的工具将简化编辑流程。

第 6 层：运行时上下文

- 如果表格缺乏先前的上下文或现有信息过时，智能体可以向数据仓库发起实时查询，以直接检查和查询该表格。这使其能够验证模式、实时理解数据，并做出相应的回复。
- 该智能体还能根据需要与其他数据平台系统（元数据服务、Airflow、Spark）进行对话，以获取数据仓库之外更广泛的数据上下文。

OpenAI

annotations, and Codex-derived enrichment into a single, normalized representation. This enriched context is then converted into embeddings using the [OpenAI embeddings API](#) and stored for retrieval. At query time, the agent pulls only the most relevant embedded context via [retrieval-augmented generation](#) (RAG) instead of scanning raw metadata or logs. This makes table understanding fast and scalable, even across tens of thousands of tables, while keeping runtime latency predictable and low. Runtime queries are issued to our data warehouse live as needed.

Together, these layers ensure the agent's reasoning is grounded in OpenAI's data, code, and institutional knowledge, dramatically reducing errors and improving answer quality.

Built to think and work like a teammate

One-shot answers work when the problem is clear, but most questions aren't. More often, arriving at the correct result requires back-and-forth refinement and some course correction.

The agent is built to behave like a teammate you can reason with. It's a conversational, always-on and handles both quick answers and iterative exploration.

It carries over complete context across turns, so users can ask follow-up questions, adjust their intent, or change direction without restating everything. If the agent starts heading down the wrong path, users can interrupt mid-analysis and redirect it, just like working with a human collaborator who listens instead of plowing ahead.

OpenAI

asks clarifying questions. If no response is provided, it applies sensible defaults to make progress. For example, if a user asks about business growth with no date range specified, it may assume the last seven or 30 days. These priors allow it to stay responsive and non-blocking while still converging on the right outcome.

The result is an agent that works well both when you know **exactly what you want** (e.g., “Tell me about this table”) and **just as strong when you’re exploring** (e.g., “I’m seeing a dip here, can we break this down by customer type and timeframe?”).

After rollout, we observed that users frequently ran the same analyses for routine repetitive work. To expedite this, the agent's workflows package recurring analyses into reusable instruction sets. Examples include workflows for weekly business reports and table validations. By encoding context and best practices once, workflows streamline repeat analyses and ensure consistent results across users.

Moving fast without breaking trust

Building an always-on, evolving agent means quality can drift just as easily as it can improve. Without a tight feedback loop, regressions are inevitable and invisible. The only way to scale capability without breaking trust is through systematic evaluation.

In this section, we’ll discuss how we leverage OpenAI’s Evals API to measure and protect the agent’s response quality.

Its Evals are built on curated sets of question-answer pairs. Each question targets an important metric or analytical pattern we care deeply about getting right, paired with a manually authored “golden” SQL query that produces the expected result. For each eval, we send the natural language question to its query-generation endpoint, execute the generated SQL, and compare the output against the result of the expected SQL.

OpenAI

Evaluation doesn't rely on naive string matching. Generated SQL can differ syntactically while still being correct, and result sets may include extra columns that don't materially affect the answer. To account for this, we compare both the SQL and the resulting data, and feed these signals into OpenAI's Evals grader. The grader produces a final score along with an explanation, capturing both correctness and acceptable variation.

These evals are like unit tests that run continuously during development to identify regressions as canaries in production; this allows us to catch issues early and confidently iterate as the agent's capabilities expand.

Agent security

Our agent plugs directly into OpenAI's existing security and access-control model. It operates purely as an interface layer, inheriting and enforcing the same permissions and guardrails that govern OpenAI's data.

All of the agent's access is strictly pass-through, meaning users can only query tables they already have permission to access. When access is missing, it flags this or falls back to alternative datasets the user is authorized to use.

最后，它的设计注重透明度。任何系统都会出错，它也不例外。它会在每个答案旁边总结假设和执行步骤，从而**公开其推理过程**。查询执行时，它会直接链接到底层结果，使用户能够查看原始数据并验证分析的每一步。

吸取的教训

从零开始构建我们的代理，让我们获得了关于代理如何运作、它们在哪些方面存在困难以及是什么真正使它们能够大规模可靠运行的实用经验。

第一课：少即是多

早期，我们将所有工具都开放给了代理，但很快就遇到了功能重叠的问题。虽然这种冗余在某些特定情况下可能有用，而且手动调用时对用户来说也更清晰，但对代理来说却会造成困惑。为了减少歧义并提高可靠性，我们限制并整合了一些工具调用。

第二课：引导目标，而非路径。

我们还发现，过于死板的提示会降低结果。虽然许多问题都具有大致相同的分析框架，但细节差异很大，因此僵化的指令常常会将智能体引向错误的路径。通过转向更高层次的指导，并依靠 GPT-5 的推理能力来选择合适的执行路径，智能体变得更加稳健，并产生了更好的结果。

第三课：意义蕴藏于代码之中

模式和查询历史描述了表的结构和用途，但其真正含义蕴藏在生成它的代码中。管道逻辑捕获了 SQL 或元数据中永远不会体现的假设、新鲜度保证和业务意图。通过使用 Codex 抓取代码库，我们的代理能够理解数据集的实际构建方式，并更好地推断每个表的实际内容。与仅依赖数据仓库信号相比，它能够更准确地回答“这里面有什么”和“我什么时候可以用它”等问题。

同样的愿景，新的工具

我们不断致力于改进我们的智能代理，提升其处理模糊问题的能力，通过更严格的验证提高其可靠性和准确性，并将其更深入地集成到工作流程中。我们相信，它应该自然地融入人们现有的工作方式，而不是像一个独立的工具。

虽然我们的工具将继续受益于代理推理、验证和自我纠正等方面的底层改进，但我们团队的使命仍然不变：在 OpenAI 的数据生态系统中无缝地提供快速、值得信赖的数据分析。

翻译反馈

此页面读起来是否轻松易懂？

😊 优秀

😞 德拉

2026

作者

Bonnie Xu、Aravind Suresh、Emma Tang

OpenAI

继续阅读

查看全部

从模型到智能体：为 Responses API 配备计算机环境

工程 2026年3月11日

超越速率限制：扩大 Codex 和 Sora 的访问规模

工程 2026年2月13日

工程技术：在智能体优先的世界中利用 Codex

工程 2026年2月11日

我们的研究

研究索引

研究概述

进修生

OpenAI for Science

经济研究

最新进展

GPT-5.3 即时版

GPT-5.3-Codex

GPT-5

法典

安全

安全措施

安全与隐私

信任与透明度

ChatGPT

探索 ChatGPT ↗

商务版

企业版

教育版

价格 ↗

下载 ↗

索拉

索拉概述

功能

价格

索拉登录 ↗

API平台

平台概览

价格

API登录 ↗

文档 ↗

开发者论坛 ↗

商业应用

商业应用概述

解决方案

联系销售团队

公司

关于我们

我们的宪章

基金会 ↗

工作机会

品牌

支持

帮助中心 ↗

更多

新闻

客户案例

直播

播客

条款与政策

使用条款

隐私政策

其他政策

OpenAI



OpenAI © 2015–2026 [管理 Cookie](#)

[🌐 中文 中国](#)